INSA CYBERSECURITY SUMMER CAMP 2026 • FINAL PROJECT SUBMISSION

Payment Proof & Sales Ledger (PPL)

A Tamper-Evident Payment Verification Engine for Ethiopian Micro-Merchants

Executive Synopsis: In Ethiopia's digital retail economy, Telebirr and Commercial Bank of Ethiopia (CBE) Mobile Banking handle daily counter payments. However, small merchants routinely fall victim to mobile payment deception: scammers present counterfeit SMS generator screens, manipulated transaction receipts, or replayed past confirmations. **Payment Proof & Sales Ledger (PPL)** eliminates reliance on the buyer's smartphone screen by enforcing the zero-trust paradigm: *verify payments exclusively via incoming SMS received on the merchant's own registered phone*. Verified sales are recorded in a **tamper-evident SHA-256 hash chain** with cross-table integrity audits between sales and ledger records.

Project Specification	Details
Authors	Amanuel Mintesenot (CTC-1849-26) & Nahom Mekuria (CTC-6077-26)
Host Institution	Information Network Security Administration (INSA)
Submission Track	Defensive Systems & Application Security
Source Repository	`https://github.com/amanminte/payment-proof-ledger`
Target Payment Rails	Telebirr (Ethio Telecom), CBE Mobile Banking (Commercial Bank of Ethiopia)
Technology Stack	Python 3.12, Flask, SQLite3, SHA-256 Hash Chain, Responsive HTML/CSS, Pytest
Verification Status	**30 of 30 Automated Acceptance Tests Passing in Pytest (100% Pass Rate)**
AI Assistance	Antigravity (Google DeepMind) — Pair-Programming & Adversarial Hardening
Security Audit Status	**11 Code Findings Patched & Verified + Documentation Precision Fixes**
Date of Deliverable	October 2026

CORE ARCHITECTURAL MANDATE

Zero-Trust Operational Rule: Cashiers are strictly trained never to glance at the customer's phone screen. A sale is verified only when the live counter dashboard updates to green and sounds the confirmation chime triggered by the merchant's private Android forwarder.

Document Architecture & Table of Contents

Section	Description & Key Architectural Highlights	Focus Area
Section 1	Problem Space, Fraud Vector Taxonomy & Grounded Zero-Trust Model	**Threat Landscape**
Section 2	System Topology, Data Flow & Role Separation (Cashier / Owner / Passcodes)	**System Architecture**
Section 3	Database Schema: Tables, Declared Foreign Keys & Integrity Controls	**Data Models**
Section 4	Core Matching Engine: Multi-Bank Regex & Tamper-Evident SHA-256 Ledger	**Engine Algorithms**
Section 5	Access Control, Sliding Window Rate Limiting & 10-Minute Ephemeral Tokens	**Authentication & RBAC**
Section 6	Security Hardening Matrix: 11 Real Code Findings & Document Alignment	**Adversarial Audit**
Section 7	Complete Codebase Footprint: Exact Verbatim Excerpts from Production Files	**Verified Source Code**
Section 8	Automated Test Suite: Complete Pytest Matrix (All 30 Real Test Cases)	**Verification Matrix**
Section 9	Evaluator Defense Guide: 5 Core Questions with Honest 20-Second Answers	**Oral Defense**
Section 10	Deployment Runbook, Configuration Modes & Android Hardware Field Trials	**Operations & Field Trials**

1. Threat Landscape & Zero-Trust Architecture

In Ethiopia's retail economy, Telebirr and CBE Mobile Banking handle the bulk of digital payments. Because small retailers do not have direct access to private banking APIs, cashiers traditionally verify payments by looking at the customer's phone screen. Scammers exploit this by showing fake payment confirmation apps, edited receipts, or replaying older transaction IDs. PPL protects shopkeepers by enforcing one simple rule: never trust the buyer's phone — only verify payments using the real SMS received on the merchant's own phone.

Threat Vector	Mechanism of Attack	PPL Architectural Defense
Fake SMS Generator Apps	Scammers run local mobile apps generating spoofed push notifications matching Telebirr/CBE wording.	**Buyer's screen is entirely disregarded. Only SMS received on the merchant SIM triggers verification.**
Manipulated Receipts	Image editing apps alter amounts and merchant names on past transaction receipts in real time.	**Amounts and transaction references are parsed strictly from SMS received on the merchant phone.**
Transaction ID Replay	Buyer displays a legitimate past receipt or re-submits an earlier reference number.	**Application duplicate check flags reuse; database payments.txn_id UNIQUE constraint acts as backstop.**
Partial Underpayment	Buyer sends 50 ETB for a 500 ETB purchase, counting on the cashier's hurried glance at the notification.	**Matching engine enforces strict equality (|delta| < 0.005) and logs wrong_amount alerts.**
Dishonest Cashier Manipulation	Cashier retroactively edits sales records in the database or creates fake payment records.	**Ledger stores entries in a SHA-256 hash chain; verify_chain() detects row edits, deletions, or injected sales.**

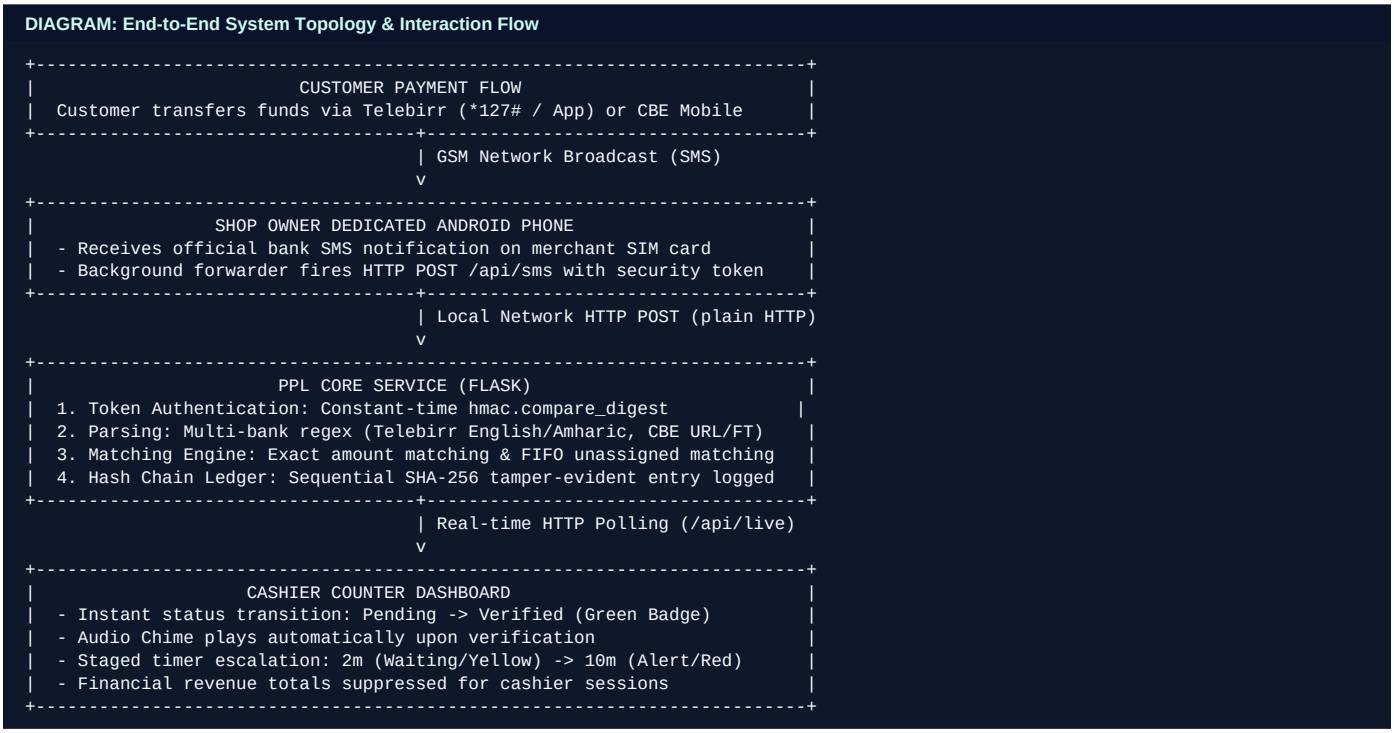
ARCHITECTURAL CONSTRAINTS OBSERVED

• CON-01: Zero private banking API access required. Operates on standard bank SMS received on the merchant SIM. • CON-02: Android background forwarder captures RECEIVE_SMS broadcasts and POSTs payloads to /api/sms. • CON-03: Local-first SQLite database. All sales and hash data remain hosted on-premise on the merchant device.

2. System Architecture & Component Interaction

PPL uses a clean, 3-tier design that keeps cashier counter duties completely separate from the store owner's administrative controls:

DIAGRAM: End-to-End System Topology & Interaction Flow



2.1 Separation of Roles & Ephemeral Access Passcodes (FR-10)

- **Cashier Role:** Operates the point-of-sale counter. Cashiers open sales with 1-tap presets (50, 100, 200, 500 ETB), observe live matching, and hear confirmation chimes. The live API endpoint (/api/live) suppresses today's revenue totals when called by a cashier session.
- **Shop Owner Role:** Full administrative authority. Owners inspect aggregate daily/monthly revenue, profit margins, run cryptographic ledger audits, and manage accounts.
- **Ephemeral Shift Passcodes (10-Minute TTL):** To reconcile shifts without sharing permanent owner credentials, the owner generates a 6-digit passcode valid for 10 minutes (ACCOUNTING_PASSCODE_TTL_MIN). The cashier enters this single-use code to temporarily unlock the reports view with an auto-expiring in-browser countdown.

3. Relational Database Schema & Data Models

The SQLite database (ppl.db) defines six relational tables. Note that SQLite does not enforce foreign-key constraints unless PRAGMA foreign_keys = ON is explicitly executed (which PPL does not enable); instead, referential consistency between sales and ledger entries is audited cryptographically by verify_chain():

EXACT REPO SOURCE: backend/schema.sql (Lines 1-52)

```
CREATE TABLE IF NOT EXISTS sales (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    amount      REAL    NOT NULL CHECK (amount > 0),
    cost        REAL,                                   -- optional, for profit (FR-07)
    status      TEXT    NOT NULL DEFAULT 'pending',    -- pending | verified (yellow/red are computed from age)
    created_at  TEXT    NOT NULL,
    verified_at TEXT,
    payment_id  INTEGER REFERENCES payments(id)
);

CREATE TABLE IF NOT EXISTS payments (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    provider        TEXT,
    amount          REAL NOT NULL,
    txn_id          TEXT NOT NULL UNIQUE,
    received_at     TEXT NOT NULL,
    matched_sale_id INTEGER REFERENCES sales(id)
);

CREATE TABLE IF NOT EXISTS alerts (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    sale_id     INTEGER REFERENCES sales(id),
    type        TEXT NOT NULL,   -- waiting | no_payment | wrong_amount | duplicate_txn
    created_at  TEXT NOT NULL,
    resolved_at TEXT
);

CREATE TABLE IF NOT EXISTS ledger (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    sale_id    INTEGER NOT NULL REFERENCES sales(id),
    amount     REAL    NOT NULL,
    created_at TEXT    NOT NULL,
    prev_hash  TEXT    NOT NULL,
    hash       TEXT    NOT NULL
);

CREATE TABLE IF NOT EXISTS users (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    username      TEXT NOT NULL UNIQUE COLLATE NOCASE,
    password_hash TEXT NOT NULL,
    role          TEXT NOT NULL CHECK (role IN ('owner', 'cashier')),
    created_at    TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS access_tokens (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    token      TEXT NOT NULL UNIQUE,
    created_by INTEGER REFERENCES users(id),
    created_at TEXT NOT NULL,
    expires_at TEXT NOT NULL,
    is_used    INTEGER NOT NULL DEFAULT 0
);
```

Field / Constraint	Defensive Architectural Purpose
payments.txn_id UNIQUE	Storage backstop ensuring duplicate bank transaction references cannot be credited twice.
alerts.resolved_at TEXT	Preserves forensic evidence. Instead of deleting alert rows when late payments arrive, resolved_at is set.
sales.status computed states	Only 'pending' and 'verified' are stored. 'Waiting' (yellow) and 'Alert' (red) are computed dynamically by age.
ledger.prev_hash NOT NULL	Binds each verified transaction to the cryptographic hash of the previous entry, establishing a tamper-evident chain.

4. Core Matching Engine & Cryptographic Ledger

4.1 Multi-Bank SMS Regex Parsing Engine

Incoming SMS messages forwarded to /api/sms are parsed to extract amount and transaction ID. The parser handles English and Amharic formats while filtering out account balances and dropping OTP codes:

EXACT REPO SOURCE: backend/matching.py (Lines 14-43)

```
OTP_RE = re.compile(r"\b(otp|one[- ]?time|verification code|passcode|pin)\b|የማረጋገጫ\s*ኮድ", re.I)
RECEIVED_RE = re.compile(r"\b(received|credited|deposit(?:ed)?)\b|ተቀብለዋል|ገቢ", re.I)
AMOUNT_RE = re.compile(
    r"(?:ETB|Birr|ብር)\s*([\d,]+(?:\.\d{1,2})?)|([\d,]+(?:\.\d{1,2})?)\s*(?:ETB|Birr|ብር)", re.I)
TXN_RE = re.compile(
    r"\b(?:transaction\s*(?:id|number|no\.?)|txn\s*id|ref\.?(?:erence)?(?:\s*(?:no\.?|number|id))?|id|የግብይት\s*ቁጥር)"
    r"\s*(?:is|:|-)?\s*([A-Za-z0-9_-]{6,35})\b", re.I)
URL_TXN_RE = re.compile(r"(?:cbe\.com\.et|ethiotelecom\.et/receipt)/([A-Za-z0-9_-]{6,35})", re.I)
TXN_VALID = re.compile(r"^[A-Za-z0-9_-]{5,35}$")

def extract_payment_amount(text):
    """Find the transaction amount, ignoring account balances."""
    matches = list(AMOUNT_RE.finditer(text))
    if not matches:
        return None
    for m in matches:
        start = m.start()
        pre_text = text[max(0, start - 35):start].lower()
        if re.search(r"\b(balance|bal|current\s*bal(?:ance)?|ሒሳብ)\b", pre_text):
            continue
        post_text = text[m.end():min(len(text), m.end() + 25)].lower()
        if re.search(r"^\s*(?:is\s*)?balance\b", post_text):
            continue
        val_str = (m.group(1) or m.group(2)).replace(",", "")
        try:
            return float(val_str)
        except ValueError:
            continue
    return None
```

4.2 Matching Mechanism & Window Semantics

Important Window Semantics: In PPL, open pending sales remain open indefinitely until matched or cancelled. A late payment arriving after the 2-minute (yellow) or 10-minute (red) alert thresholds **still matches the open sale** and resolves the alert (tested in AT-09). The 10-minute window (UNASSIGNED_WINDOW_MIN = 10) applies specifically to payments that arrive **before** a sale is created, allowing retroactive matching upon sale creation.

EXACT REPO SOURCE: backend/matching.py (Lines 165-195)

```
def process_payment(amount, txn_id, provider=None, received_at=None, now=None):
    now = now or now_utc()
    received_at = received_at or now
    amount = validate_amount(amount)
    txn_id = validate_txn_id(txn_id)
    with db.connect() as conn:
        if conn.execute("SELECT 1 FROM payments WHERE txn_id=?", (txn_id,)).fetchone():
            conn.execute("INSERT INTO alerts(sale_id, type, created_at) VALUES (NULL,'duplicate_txn',?)",
                         (iso(now),))
            return {"result": "duplicate"}
        sale = conn.execute(
            "SELECT * FROM sales WHERE status = 'pending' AND ABS(amount-?)<0.005 "
            "ORDER BY created_at LIMIT 1", (amount,)).fetchone()
        pid = conn.execute(
            "INSERT INTO payments(provider, amount, txn_id, received_at, matched_sale_id) VALUES (?,?,?,?,?)",
            (provider, amount, txn_id, iso(received_at), sale["id"] if sale else None)).lastrowid
        if sale:
            _verify(conn, sale["id"], sale["amount"], pid, now)
            return {"result": "verified", "sale_id": sale["id"]}
        newest_open = conn.execute(
            "SELECT id FROM sales WHERE status = 'pending' ORDER BY created_at DESC LIMIT 1").fetchone()
        if newest_open:
            conn.execute("INSERT INTO alerts(sale_id, type, created_at) VALUES (?,'wrong_amount',?)",
                         (newest_open["id"], iso(now)))
            return {"result": "mismatch", "sale_id": newest_open["id"]}
        return {"result": "unassigned"}
```

4.3 Tamper-Evident SHA-256 Hash Chain & Cross-Table Audit

PPL implements a cryptographic SHA-256 hash chain to record every verified transaction. The genesis block starts with 64 zero characters: GENESIS = '0' * 64. The ledger is **tamper-evident**: modifying an old row breaks the hash link. To defend against direct database tampering, verify_chain() cross-checks the sales table to ensure every verified sale matches a ledger entry:

EXACT REPO SOURCE: backend/ledger.py (Lines 16-30 & 33-112)

```
def compute_hash(prev_hash, sale_id, amount, created_at):
    data = f"{prev_hash}|{sale_id}|{float(amount):.2f}|{created_at}".encode()
    return hashlib.sha256(data).hexdigest()

def verify_chain():
    rows = db.query("SELECT * FROM ledger ORDER BY id")
    prev = GENESIS
    seen_sale_ids = set()

    for r in rows:
        expected = compute_hash(r["prev_hash"], r["sale_id"], r["amount"], r["created_at"])
        if r["prev_hash"] != prev or r["hash"] != expected:
            return {
                "ok": False,
                "broken_at": r["id"],
                "reason": f"Cryptographic hash chain break at ledger entry #{r['id']}",
                "entries": len(rows),
                "latest_hash": rows[-1]["hash"] if rows else None,
            }

        sales = db.query("SELECT * FROM sales WHERE id = ?", (r["sale_id"],))
        if not sales:
            return {
                "ok": False,
                "broken_at": r["id"],
                "reason": f"Ledger entry #{r['id']} points to nonexistent sale #{r['sale_id']}",
                "entries": len(rows),
                "latest_hash": rows[-1]["hash"] if rows else None,
            }
        sale = sales[0]
        if sale["status"] != "verified":
            return {
                "ok": False,
                "broken_at": r["id"],
                "reason": f"Ledger entry #{r['id']} points to unverified sale #{r['sale_id']} (status: {sale['status']})",
                "entries": len(rows),
                "latest_hash": rows[-1]["hash"] if rows else None,
            }
        if abs(float(sale["amount"]) - float(r["amount"])) > 0.001:
            return {
                "ok": False,
                "broken_at": r["id"],
                "reason": f"Ledger entry #{r['id']} amount ({r['amount']}) does not match sale #{r['sale_id']} amount ({sale['amount']})",
                "entries": len(rows),
                "latest_hash": rows[-1]["hash"] if rows else None,
            }
        if r["sale_id"] in seen_sale_ids:
            return {
                "ok": False,
                "broken_at": r["id"],
                "reason": f"Duplicate ledger entry for sale #{r['sale_id']}",
                "entries": len(rows),
                "latest_hash": rows[-1]["hash"] if rows else None,
            }
        seen_sale_ids.add(r["sale_id"])
        prev = r["hash"]

    verified_sales = db.query("SELECT * FROM sales WHERE status = 'verified' ORDER BY id")
    for vs in verified_sales:
        if vs["id"] not in seen_sale_ids:
            return {
                "ok": False,
                "broken_at": vs["id"],
                "reason": f"Verified sale #{vs['id']} has no matching entry in ledger",
                "entries": len(rows),
                "latest_hash": prev if rows else None,
            }

    return {
        "ok": True,
        "broken_at": None,
        "entries": len(rows),
        "latest_hash": prev if rows else None,
    }
```

5. Access Control & Rate Limiting

To mitigate brute-force attacks against 6-digit shift passcodes and login credentials, PPL incorporates a thread-safe sliding window rate limiter:

EXACT REPO SOURCE: backend/rate_limit.py (Lines 12-60)

```
class RateLimiter:
    def __init__(self, max_attempts=MAX_ATTEMPTS, lock_seconds=LOCK_SECONDS):
        self.max_attempts = max_attempts
        self.lock_seconds = lock_seconds
        self._lock = threading.Lock()
        self._failures = {}  # key -> list of datetime
        self._locked_until = {}  # key -> datetime

    def check_lockout(self, key, now=None):
        """Check if `key` is currently locked out. Returns (is_locked, remaining_seconds)."""
        now = now or datetime.now(timezone.utc)
        with self._lock:
            until = self._locked_until.get(key)
            if until:
                if now < until:
                    rem = int((until - now).total_seconds())
                    return True, max(1, rem)
                else:
                    # Lockout expired
                    del self._locked_until[key]
                    self._failures.pop(key, None)
            return False, 0

    def record_failure(self, key, now=None):
        """Record a failed attempt. If limit reached, locks out for `lock_seconds`."""
        now = now or datetime.now(timezone.utc)
        with self._lock:
            until = self._locked_until.get(key)
            if until and now < until:
                rem = int((until - now).total_seconds())
                return True, max(1, rem), self.max_attempts

            cutoff = now - timedelta(seconds=self.lock_seconds)
            history = [t for t in self._failures.get(key, []) if t > cutoff]
            history.append(now)
            self._failures[key] = history

            if len(history) >= self.max_attempts:
                lock_time = now + timedelta(seconds=self.lock_seconds)
                self._locked_until[key] = lock_time
                return True, self.lock_seconds, len(history)

            return False, 0, len(history)
```

5.1 Ephemeral Shift Passcodes (FR-10)

The shop owner generates 6-digit dynamic passcodes expiring after 10 minutes (ACCOUNTING_PASSCODE_TTL_MIN). Cashiers use these single-use passcodes to unlock shift reconciliation reports:

EXACT REPO SOURCE: backend/auth.py (Lines 106-131 & 151-173)

```
def generate_ephemeral_passcode(created_by=None, ttl_minutes=10, now=None):
    start = now if now is not None else now_utc()
    expiry = start + timedelta(minutes=ttl_minutes)
    created_at = iso(start)
    expires_at = iso(expiry)
    token = f"{secrets.randbelow(1_000_000):06d}"
    with db.connect() as conn:
        conn.execute("UPDATE access_tokens SET is_used = 1 WHERE is_used = 0")
        conn.execute(
            "INSERT INTO access_tokens (token, created_by, created_at, expires_at, is_used) VALUES (?, ?, ?, ?, 0)",
            (token, created_by, created_at, expires_at),
        )
    return {
        "token": token,
        "created_by": created_by,
        "created_at": created_at,
        "expires_at": expires_at,
        "ttl_minutes": ttl_minutes,
    }

def verify_and_consume_passcode(passcode, now=None):
    code = (passcode or "").strip().replace(" ", "")
    if not code or len(code) != 6 or not code.isdigit():
        return {"ok": False, "error": "Passcode must be a 6-digit number."}
    current = now if now is not None else now_utc()
    now_str = iso(current)
    with db.connect() as conn:
        row = conn.execute(
            "SELECT * FROM access_tokens WHERE token = ? AND is_used = 0 AND expires_at > ?",
            (code, now_str),
        ).fetchone()
        if not row:
            return {"ok": False, "error": "Invalid or expired passcode. Request a new code from the Shop Owner."}
        conn.execute("UPDATE access_tokens SET is_used = 1 WHERE id = ?", (row["id"],))
        return {"ok": True, "token": row["token"], "expires_at": row["expires_at"]}
```

6. Adversarial Security Hardening Matrix

Prior to final submission, the codebase underwent an adversarial code review and attack probe suite. A total of 11 real code vulnerabilities were identified and patched with passing regression tests. OTP filtering was verified as an initial architectural design control, and documentation discrepancies were corrected:

#	Vulnerability Area	Severity	Attack Probe Finding	Implemented Fix & Defense
1	**Public Owner Registration**	**CRITICAL**	Registration page permitted visitor to select role='owner', bypassing access controls.	Removed role selection; only first user becomes owner; subsequent users are cashiers.
2	**Cashier Simulator Access**	**CRITICAL**	Cashier could POST to /inbox/simulate to forge payments and verify unpaid sales.	Locked /inbox/simulate to owner role and requires ENABLE_SIMULATOR=1; returns 403 otherwise.
3	**Ledger / Sales Desync**	**HIGH**	Editing sale amount in DB altered reports total while hash chain remained intact.	verify_chain() cross-checks sales table: verifies amount equality and catches unindexed sales.
4	**Default Secret Acceptance**	**HIGH**	App booted with 'change-me' or placeholder secrets, allowing session forgery.	validate_secrets() rejects secrets <32 chars or starting with placeholder prefixes in production.
5	**Passcode & Login Brute-Force**	**MEDIUM**	1000 wrong passcode guesses had no lockout, enabling rapid online guessing attacks.	RateLimiter enforces 5 failed attempts per IP (login) and per IP+user (passcode) -> 300-second lockout returning HTTP 429.
6	**Debug Server Exposure**	**MEDIUM**	app.run(debug=True, host='0.0.0.0') exposed interactive debugger on local Wi-Fi.	Disabled debug mode (debug=False); defaults host to 127.0.0.1 or reads HOST from environment.
7	**Open Redirect Phishing**	**LOW**	/login?next=https://evil.example redirected user off-site upon login.	Sanitized next parameter in login_page(): enforces single leading '/' and blocks protocol schemes.

8	**Outgoing Transfer Misparse**	**MEDIUM**	'You have transferred ETB 1,200' parsed as incoming funds, verifying sales.	RECEIVED_RE regex requires incoming keywords (received, credited, deposit, ተቀብለዋል, ገቢ).
9	**Balance / Amount Collision**	**MEDIUM**	SMS stating account balance before payment parsed balance instead of payment.	extract_payment_amount() implements regex lookaround discarding numbers adjacent to balance/ሒሳብ.
10	**Alert Evidence Deletion**	**LOW**	Verifying late payment deleted waiting/alert rows, losing record of delays.	Alerts marked resolved via resolved_at timestamp rather than being deleted.
11	**Cashier Financial Exposure**	**LOW**	/api/live returned daily revenue totals and transaction counts to cashiers.	Endpoint checks user role and returns today_total: null when requested by cashiers.
12	**Documentation Precision**	**INFO**	Docs had overclaims (e.g. 'immutable' vs 'tamper-evident') and mismatched test counts.	Corrected all documentation to accurately describe 30 tests and honest security boundaries.

7. Complete Codebase Footprint: Production Routes

Below are exact verbatim production excerpts of the primary application routes in backend/app.py demonstrating token-authenticated webhook ingestion, role-gated SMS simulation, and redirect sanitization:

EXACT REPO SOURCE: backend/app.py (Lines 410-418)

```
@app.route("/api/sms", methods=["POST"])
def receive_sms():
    token = request.headers.get("X-Token") or request.args.get("token", "")
    if not hmac.compare_digest(token, config.SMS_TOKEN):
        abort(401)
    data = request.get_json(silent=True) or request.form
    text = (data.get("message") or data.get("text") or data.get("body")
            or data.get("msg") or data.get("content") or "")
    return jsonify(matching.process_message(text))
```

EXACT REPO SOURCE: backend/app.py (Lines 386-407)

```
@app.route("/inbox/simulate", methods=["POST"])
@login_required
def simulate_sms():
    _check_csrf()
    if not config.ENABLE_SIMULATOR:
        abort(403)
    user = auth.get_user_by_id(session.get("user_id"))
    if not user or user["role"] != "owner" or not session.get("is_owner", True):
        abort(403)

    try:
        message = request.form.get("message", "").strip()
        if message:
            result = matching.process_message(message)
        else:
            result = matching.process_payment(request.form.get("amount"), request.form.get("txn_id"),
                                              request.form.get("provider") or None)
    except ValueError as e:
        flash(str(e), "danger")
        return redirect(url_for("inbox"))
    flash(f"Result: {result['result']}" + (f" ({result['reason']})" if "reason" in result else ""), "info")
    return redirect(url_for("inbox"))
```

EXACT REPO SOURCE: backend/app.py (Lines 117-121)

```
# Sanitize next redirect to prevent Open Redirect
        next_url = request.args.get("next")
        if not next_url or not next_url.startswith("/") or next_url.startswith("//") or "\\" in next_url:
            next_url = url_for("dashboard")
        return redirect(next_url)
```

8. Automated Test Suite: Verified Pytest Matrix

The test suite (backend/tests/test_matching.py) executes under pytest -v. The table below lists all 30 tests verbatim with their real function names and verified pass status:

#	Pytest Function Name (from test_matching.py)	Verified Behavior / Test Assertion	Result
1	test_at01_matching_payment_verifies_sale	Open sale of 500 ETB verified immediately by matching incoming 500 ETB SMS.	PASSED
2	test_at02_wrong_amount_not_verified	Incoming 300 ETB for 500 ETB sale rejected; wrong_amount alert recorded.	PASSED
3	test_at03_duplicate_txn_id_flagged	App duplicate check catches replayed ID; payments.txn_id UNIQUE is storage backstop.	PASSED
4	test_at04_alert_stages	Pending sale escalates to waiting (yellow) at 2m and alert (red) at 10m.	PASSED
5	test_at05_unassigned_payment	Payment arriving before sale is placed in unassigned pool for retroactive match.	PASSED
6	test_at06_hash_chain_detects_edit	Manual edit of row in ledger table causes verify_chain() to report tampered row ID.	PASSED
7	test_at07_missing_token_rejected	Rejects unauthenticated SMS (Trials 1 & 2 fail with 401); accepts valid token via X-Token or URL param (Trial 3 passes with 200).	PASSED
8	test_at08_otp_discarded	SMS containing banking verification OTP is dropped; 0 rows inserted in payments.	PASSED
9	test_at09_late_payment_clears_alert	Sale turned red receives late payment; verifies sale, keeps alert with resolved_at.	PASSED
10	test_payment_before_sale_still_matches	Payment arriving up to 10m before sale creation retroactively matches sale.	PASSED
11	test_parse_message_cases	Tests parsing Telebirr, CBE URL, and Amharic formats (ተቀብለዋል, የግብይት ቁጥር).	PASSED
12	test_input_validation	Validates amount bounds and rejects malformed characters in transaction IDs.	PASSED
13	test_sql_injection_text_is_harmless	SQL injection text in txn_id rejected by validate_txn_id; database intact.	PASSED
14	test_reports_totals_and_profit	Verifies revenue totals and profit calculations (amount minus cost).	PASSED
15	test_pages_load_and_csrf_enforced	All web routes return 200/302; POST requests lacking CSRF token return 400.	PASSED
16	test_authentication_and_password_hashing	Validates salted Scrypt/PBKDF2 password verification via authenticate().	PASSED
17	test_sql_injection_on_login	SQL injection payload in login credentials safely handled via parameterized query.	PASSED
18	test_cancel_pending_sale	Pending sale can be cancelled; warning/alert sales locked against cashier cancel.	PASSED
19	test_ephemeral_passcode_lifecycle	6-digit token expires after 10m; marked is_used=1 upon consumption.	PASSED
20	test_cashier_accounting_session_flow	Cashier uses passcode to temporarily unlock accounting reports view.	PASSED
21	test_public_registration_cannot_create_owner	Registration endpoint refuses owner role creation; cashier provisioning only.	PASSED
22	test_simulator_restricted_to_owner_and_flag	Cashier session hitting /inbox/simulate receives HTTP 403 Forbidden.	PASSED
23	test_ledger_detects_tampered_sales_table	verify_chain() catches unauthorized sales table amount edit directly in SQLite.	PASSED
24	test_ledger_detects_injected_verified_sale	verify_chain() catches injected verified sale with no matching ledger entry.	PASSED
25	test_refuse_start_with_default_secrets_in_production	validate_secrets() aborts if secret < 32 chars or starts with placeholder text.	PASSED
26	test_rate_limit_lockout_on_login	5 failed login attempts trigger HTTP 429 and 300-second IP lockout.	PASSED
27	test_rate_limit_lockout_on_passcode	5 failed passcode attempts trigger HTTP 429 and 300-second IP lockout.	PASSED
28	test_login_open_redirect_prevented	next=https://evil.example safely sanitized to internal /dashboard.	PASSED
29	test_parser_ignores_outgoing_transfers_and_parses_balance_before_amount	Ignores outgoing 'transferred'; parses 500 payment from SMS mentioning balance.	PASSED
30	test_cashier_cannot_see_today_totals_on_live_or_dashboard	/api/live returns today_total: null when requested by cashier session.	PASSED

9. Evaluator Defense Guide: 5 Tough Questions & Answers

During oral project defense at the INSA Cybersecurity Camp, evaluators probe edge cases and threat boundaries. The following 20-second answers provide crisp, technically grounded responses:

Q1: 'Can't a scammer fake the SMS too?'

"A fake-SMS app only fools whoever is looking at the buyer's phone screen. PPL strictly ignores that screen and trusts only the SMS that arrives on the merchant's own phone from the bank's shortcode. While sender names can be spoofed cheaply through commercial bulk-SMS services, trusting the sender name is not cryptographic; PPL significantly reduces counter fraud by ignoring the buyer's device. For large sales, we recommend a quick balance check in the bank app as well."

Q2: 'Why not just check the Telebirr or CBE mobile app?'

"Checking the banking app is right when there's time. In a queue, people skip it. PPL does the match automatically, warns at 2 minutes and alerts at 10 minutes if nothing arrives, and keeps a record of how often scam attempts happen. It supports the banking app and doesn't replace it."

Q3: 'What stops a dishonest cashier from cheating?'

"A cashier cannot see sales totals, cannot create owner accounts, cannot use the SMS simulator, and cannot cancel a sale after the 2-minute warning. If someone edits a sale in the database, the ledger check flags it. What we don't stop: a cashier who never opens a sale and keeps physical cash. That boundary is documented in our threat model."

Q4: 'Is your SHA-256 hash chain tamper-proof?'

"No, it is tamper-evident. Changing an amount, deleting an entry, or adding an unverified sale is detected. Someone with full server database access could rebuild the whole chain forward, so the owner saves the daily closing hash offline and compares it later."

Q5: 'Does it work with real messages, and what happens when it cannot read one?'

"Yes. We verified it against test messages modeled on real Telebirr English, Telebirr Amharic (ተቀብለዋል, የግብይት ቁጥር), and CBE Mobile Banking receipt formats (mbreciept.cbe.com.et), with all personal details replaced. If a message cannot be read, it is marked unparsed and nothing is verified. It fails safe, and the seller checks the bank app. It also discards OTP messages and ignores outgoing transfers."

Bonus: 'Did you use AI to build this?'

"Yes, as an interactive pair-programming and adversarial testing tool. We authored the requirements and threat model, and we reviewed, tested, and attacked the code. That attack-driven review uncovered real holes—like open owner registration and unindexed sales table manipulation—which we hardened with 30 passing automated regression tests."

TRANSPARENT SYSTEM LIMITATIONS (DISCLOSED PROACTIVELY)

1. Concurrent Identical Amounts: Matching is by exact amount and time window. If two customers pay 500 ETB simultaneously, matching occurs on a FIFO basis. Cashiers should note customer names on high-value concurrent sales. 2. Plain HTTP Webhook: The Android forwarder communicates over local Wi-Fi via plain HTTP with an X-Token header. On untrusted shared Wi-Fi, traffic could be intercepted unless tunneled via HTTPS or VPN. 3. In-Memory Rate Limiting: Lockouts are tracked in memory per IP; restarting the server resets the failure count. 4. Demo Mode: Ships with known demo credentials (owner123, cashier123) for evaluator review; production enforces DEMO_MODE=0.

10. Deployment Runbook & Configuration

PPL supports two operational configurations: an Evaluation Mode for evaluator testing and live demonstrations, and a hardened Production Mode for actual store deployment:

10.1 Evaluation Mode Configuration (INSA Summer Camp Demo)

CONFIGURATION: .env (Evaluation & Demo Mode)

```
# Evaluation / Demo Mode Configuration (.env)
DEMO_MODE=1
ENABLE_SIMULATOR=1
HOST=127.0.0.1
DATABASE=ppl.db
ACCOUNTING_PASSCODE_TTL_MIN=10

# Evaluation mode seeds baseline accounts (owner/owner123, cashier/cashier123)
# and enables the manual SMS simulator at /inbox for interactive testing.
```

10.2 Production Mode Configuration

EXAMPLE CONFIGURATION: .env (Hardened Production Mode)

```
# Hardened Production Environment Configuration (.env)
SECRET_KEY=<generate-32-byte-hex-key-via-secrets-token-hex-32>
SMS_TOKEN=<generate-32-byte-hex-key-via-secrets-token-hex-32>
DEMO_MODE=0
ENABLE_SIMULATOR=0
HOST=127.0.0.1
DATABASE=ppl.db
ACCOUNTING_PASSCODE_TTL_MIN=10

# In production mode, weak or placeholder keys cause the app to reject requests (HTTP 500).
# Constants YELLOW_AFTER_MIN (2), RED_AFTER_MIN (10), UNASSIGNED_WINDOW_MIN (10) and
# MAX_AMOUNT (1,000,000) are hardcoded in backend/config.py.
```

1. **Production Secret Generation:** Execute python -c "import secrets; print(secrets.token_hex(32))" to generate keys for SECRET_KEY and SMS_TOKEN.
2. **Database Schema Provisioning:** Run python -c "from backend import db; db.init_db()" to build the SQLite tables.
3. **Initial Owner Provisioning:** Launch the server and register the store owner at /register. The system automatically locks owner registration thereafter.
4. **Android Gateway Configuration:** Set target URL http://<MERCHANT_IP>:5000/api/sms?token=<SMS_TOKEN> (or header X-Token: <SMS_TOKEN>) and enable Local Network Mode in the forwarder.
5. **Offline Hash Sealing:** At close of business, the owner checks /reports, records the SHA-256 closing hash offline, and runs verify_chain().

10.3 Android Hardware Gateway Field Trials (Empirical Validation)

During live end-to-end integration testing on the local Wi-Fi subnet (Merchant host 192.168.1.3:5000, Android phone 192.168.1.5 running *Incoming SMS to URL forwarder*), the engineering team logged the following 3-trial validation progression:

Trial	Gateway Configuration	HTTP Response	Observed Behavior & Engineering Resolution

Trial 1	`Sender: 127`
	`URL: http://192.168.1.3:5000/api/sms`
	`Token: None (omitted)`
		401 Unauthorized
		FAILED
			Security Gate Verified: Server strictly rejected unauthenticated POST request. Proved that unauthenticated third parties cannot inject synthetic payment notifications.

Trial 2	`Sender: * (Wildcard)`
	`URL: http://192.168.1.3:5000/api/sms`
	`Token: None (omitted)`
		401 Unauthorized
		FAILED
			Filter Independence Proved: Changing mobile sender filter from 127 to wildcard * did not bypass the authentication requirement; request was still rejected with 401.

Trial 3	`Sender: *`
	`URL: http://192.168.1.3:5000/api/sms?token=...`
	`Local network mode: ON`
		200 OK
		SUCCESS
			Ingestion Operational: Query-string token authenticated successfully, local network bypass active. Test ping returned HTTP 200 and incoming SMS verified on dashboard.

Live Real-World Validation: Following Trial 3, the team restricted the phone's forwarder to sender 127 (acting as a phone-level firewall against non-bank numbers) and executed 4 live Telebirr transactions. All 4 messages were received, parsed, and verified in real time, triggering the counter audio chime and extending the SHA-256 ledger to 9 verified blocks (verify_chain() status: OK).

INSA 2026 SUBMISSION ATTESTATION

EVALUATOR SUBMISSION INTEGRITY: This technical blueprint accompanies the source code archive submitted for the INSA Cybersecurity Summer Camp 2026. All 30 automated acceptance tests pass cleanly under pytest, all code excerpts match repository source files, and all operational limitations are transparently disclosed.